

Vérification Formelle d'Applications Critiques avec Réseaux de Petri

Modélisation du système ERTMS/ETCS

Smilianitch Maëlle, Hure Mathieu, Pham Edouard et Bouayad Sirine

Introduction et Enjeux

Le problème : Dans l'ingénierie critique (aviation, ferroviaire), les tests classiques ne suffisent pas. Une faille = danger mortel.

La solution : Prouver mathématiquement l'absence de faille.

Deux phases distinctes :

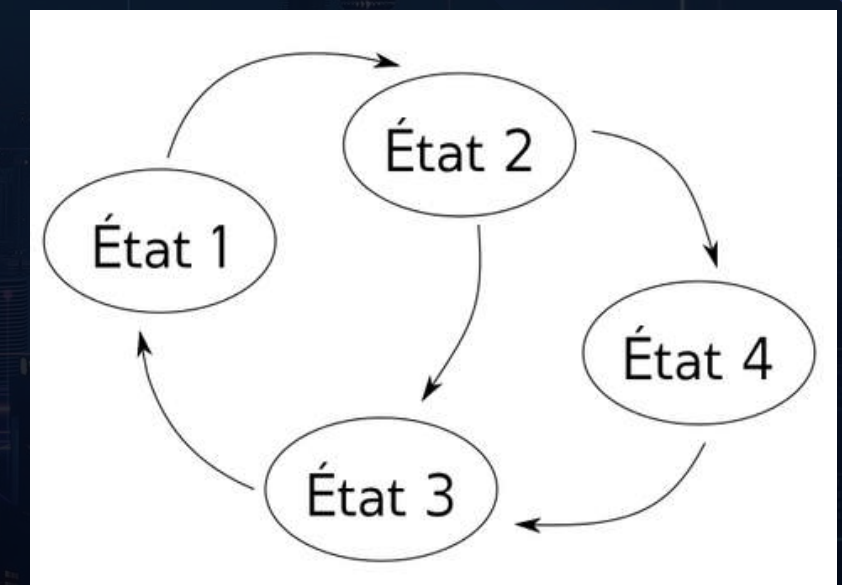
1. **Modélisation** : Langage mathématique pour décrire le système sans ambiguïté.
2. **Vérification** : Algorithmes (LTL/CTL, Model Checking) pour prouver la sûreté du modèle.

Les Modèles basés sur les Automates

Le concept : Système de "Machine à états". Séquentiel : un seul état possible à l'instant t .

FSM (Finite State Machines) / Automates à États Finis :

- **Force** : Parfait pour les protocoles stricts
- **Limite** : "Aveugle" au temps ; gère très mal le parallélisme (explosion combinatoire)



Automates Temporisés :

- **Force** : Intègre des "horloges" pour le Temps Réel Strict
- **Limite** : Mathématiquement très lourd à vérifier

Les Algèbres de Processus (CSP, CCS)

Le concept : Langage algébrique pour décrire l'interaction entre entités indépendantes

Avantage majeur : L'outil parfait pour sécuriser les protocoles de communication (ex: radio GSM-R train-sol)

Limites critiques :

- Manque de représentation physique et visuelle
- Très inadapté pour modéliser la cinématique ou le mouvement d'un objet (ex: le freinage d'un train)

Les Réseaux de Petri (RdP) : Le choix optimal

Le concept : Le système peut être dans plusieurs états simultanément.

Mécanique : Places (états), Jetons (conditions), Transitions (actions).

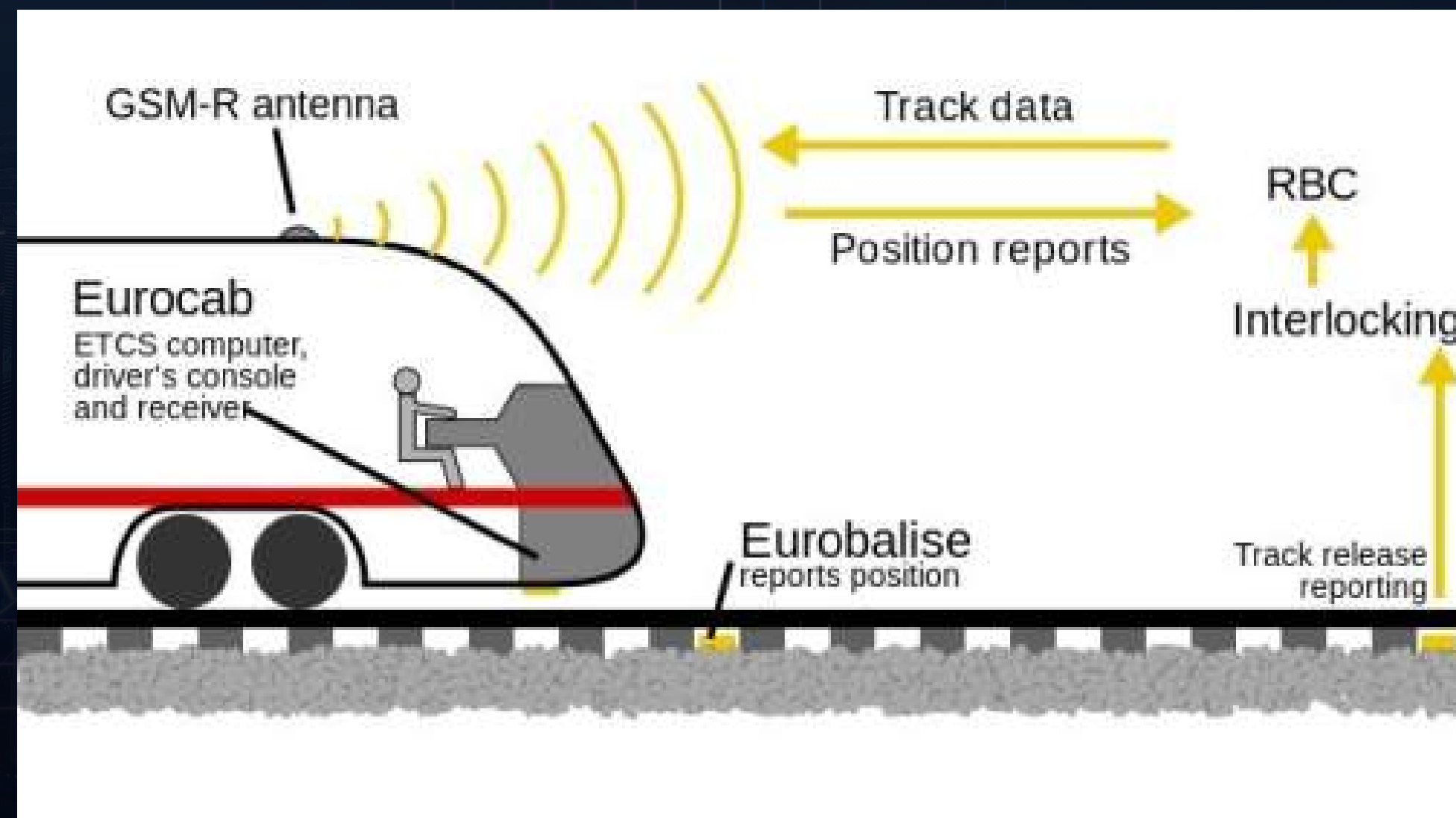
Pourquoi ce choix pour notre projet ?

- Gestion native du parallélisme.
- Modélise parfaitement l'exclusion mutuelle et le partage de ressources physiques partagées.

Les RdP Colorés : Ajouter des "couleurs" (types de données) aux jetons pour compacter les modèles complexes (ex: ID d'un train).

Choix d'une application critique

Le système ferroviaire ERTMS/ETCS Niveau 2



Sûreté "Fail-Safe" & Criticité (SIL 4)

- **Exigence industrielle** : Norme SIL 4
- **Logique "Fail-Safe" (Sûreté intégrée)** : Préférer immobiliser le train en sécurité plutôt que de le laisser rouler à l'aveugle

Composant	Risque identifié	Conséquence
RBC (Sol)	Double autorisation donnée	Collision frontale/rattrapage
GSM-R (Radio)	Perte de liaison prolongée	Roulage à l'aveugle
EVC (Bord)	Blocage logiciel (Deadlock)	Incapacité de freiner

L'Abstraction (Le passage au discret)

Le Piège : Physique continue (km/h, distance) => Explosion combinatoire

Notre Stratégie : Discrétisation booléenne de l'EVC

3 Modules :

1. **Cinématique** (Marche / Alerte / Freinages)
2. **Réseau** (Radio OK / Perdue)
3. **Localisation** (Odométrie OK / Dérive)

L'Architecture de l'Automate EVC

Le Cerveau du train (EVC) :

- 9 États du système (Places)
- 16 Événements (Transitions)

3 Modules :

1. Cinématique
2. Communication GSM-R
3. Localisation (Odométrie)

Les Modules Capteurs (Les Entrées)

Module Communication :

- Radio_OK vs Radio_Perdue
- **Aléa** : Tunnel, panne d'antenne

Module Localisation :

- Odometrie_OK vs Derive_Odo
- **Aléa** : Patinage de la roue (train "perdu")

Le Module Cinématique (La Réaction)

Dynamique de mouvement :

- Marche_Normale -> État nominal
- Alerte_Vitesse -> Dépassement de seuil
- Freinage_Service -> Freinage classique
- Freinage_Urgence -> Arrêt magnétique

Conception robuste : Modèle réversible (pas de deadlock factice).
Reprise possible si les conditions sont rétablies.

La Mécanique "Fail-Safe" (Sûreté absolue)

Exemple de transition critique : T_Urgence_Radio_Marche

La Condition : Train en marche ET Radio coupée

L'Action informatique : Le système "arrache" violemment le jeton de la marche normale

Le Résultat : Basculement forcé et inconditionnel en Freinage_Urgence

```
30 # === MECANISMES DE FAIL-SAFE ===
31 tr T_Urgence_Radio_Marche Marche_Normale Radio_Perdue ->
    Freinage_Urgence Radio_Perdue
32 tr T_Urgence_Radio_Alerte Alerte_Vitesse Radio_Perdue ->
    Freinage_Urgence Radio_Perdue
33 tr T_Urgence_Radio_Service Freinage_Service Radio_Perdue ->
    Freinage_Urgence Radio_Perdue
```


L'Astuce de Modélisation (La mémoire de panne)

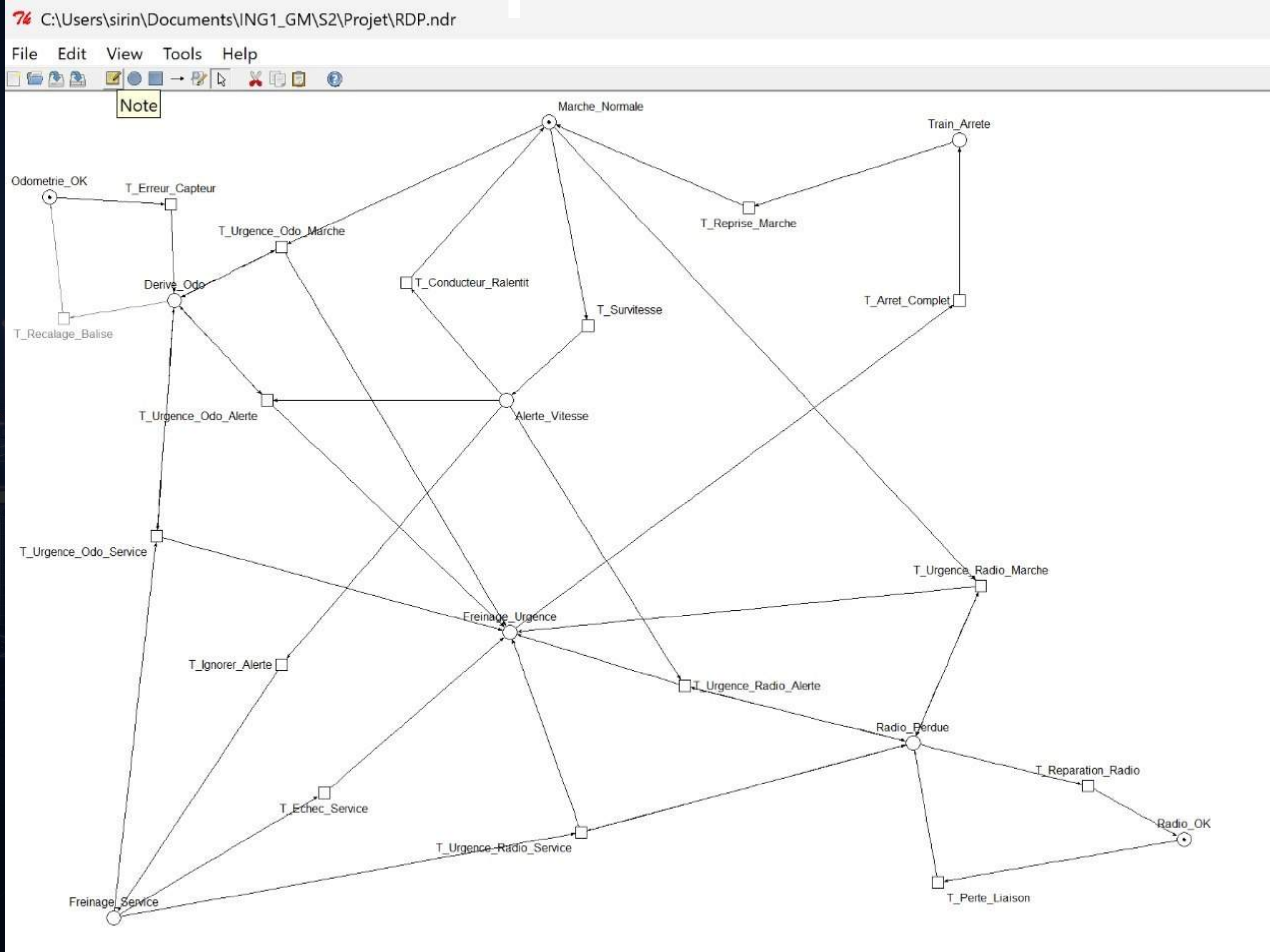
Le problème du Réseau de Petri : Une transition consomme un jeton et le détruit

Notre solution : La transition d'urgence avale le jeton de panne ET le recrache instantanément

L'impact physique : L'EVC garde la "mémoire" de la panne

Sécurité garantie : Redémarrage physiquement impossible tant que la panne n'est pas traitée

Le Modèle Complet



Le Langage de la Preuve (LTL & CTL)

L'Objectif : Traduire le cahier des charges en équations mathématiques

Les 2 Formalismes :

- **Logique LTL :** Vérifie un chemin temporel unique (linéaire)
- **Logique CTL :** Vérifie un arbre d'embranchements possibles

Les Opérateurs Clés :

- **A :** Absolument tous les chemins
- **G :** Globalement (tout le temps)
- **F :** Finalement (un jour dans le futur)
- **EX :** Il existe une étape suivante

Sécurité et "Fail-Safe" (Propriétés P1 & P2)

P1. Sécurité (Exclusion mutuelle) :

- **Exigence** : Une situation physiquement impossible ne peut jamais se produire
- **LTL** : $G (\neg(\text{Marche_Normale} \wedge \text{Freinage_Urgence}))$
- **CTL** : $AG (\neg(\text{Marche_Normale} \wedge \text{Freinage_Urgence}))$

P2. Réactivité (Sûreté intégrée)

- **Exigence** : Panne matérielle en roulage = Arrêt inconditionnel.
- **LTL** : $G ((\text{Marche_Normale} \wedge \text{Radio_Perdue}) \rightarrow F \text{Freinage_Urgence})$
- **CTL** : $AG ((\text{Marche_Normale} \wedge \text{Radio_Perdue}) \rightarrow AF \text{Freinage_Urgence})$

Continuité et Stabilité du Système

P3. Invariants (Ressources matérielles)

- **Exigence** : Un équipement a un seul état à la fois et ne disparaît jamais
- **Formules** : $G ((\text{Radio_OK} + \text{Radio_Perdue}) = 1)$ et $G ((\text{Odometrie_OK} + \text{Derive_Odo}) = 1)$

P4. Vivacité (Liveness)

- **Exigence** : Un freinage d'urgence se termine toujours par l'immobilisation
- **LTL** : $G (\text{Freinage_Urgence} \rightarrow F \text{Train_Arrete})$
- **CTL** : $AG (\text{Freinage_Urgence} \rightarrow AF \text{Train_Arrete})$

P5. Absence d'interblocage (Deadlock-Free)

- **Exigence** : Le système ne doit jamais se bloquer (geler)
- **CTL** : $AG (EX \text{ true})$

Architecture P00

Approche : Programmation Orientée Objet (POO) en Python

Concepts clés :

- **Place** : Le "vigile" (vérifie les pré-conditions)
- **Transition** : L'action (soustrait et additionne les jetons)


```
# DÉFINITION D'UNE "PLACE" (Les cercles du réseau)
class Place: 4 usages

    # La fonction __init__ est le constructeur. Elle s'exécute quand on crée une nouvelle place.
    def __init__(self, name: str, initial_tokens: int = 0):
        self.name = name # Le nom de la place (ex: "Marche_Normale")
        self.initial_tokens = initial_tokens # Le nombre de jetons qu'on y met au tout début
        self.tokens = initial_tokens # Le nombre de jetons actuels (qui va évoluer pendant la simulation)

    # Fonction pour afficher joliment la place dans la console si on l'imprime
    def __repr__(self):
        return f"Place({self.name}, tokens={self.tokens})"
```

```
# DÉFINITION D'UNE "TRANSITION" (Les rectangles / portes tournantes du réseau)
class Transition: 2 usages

    def __init__(self, name: str):
        self.name = name
        # Dictionnaires (collections de paires Clé/Valeur) pour stocker les liaisons
        self.input_arcs: Dict[Place, int] = {} # Stocke les places AVANT la transition et le nombre de jetons requis
        self.output_arcs: Dict[Place, int] = {} # Stocke les places APRÈS la transition et le nombre de jetons à créer

    # Fonction pour vérifier si la transition a le droit de s'activer ("tirer")
    def is_enabled(self) -> bool: 3 usages
        # On parcourt toutes les places d'entrée reliées à cette transition
        for place, weight in self.input_arcs.items():
            # S'il manque des jetons dans au moins une place, la transition est bloquée (False)
            if place.tokens < weight:
                return False

        # Si la boucle se termine sans problème, c'est que toutes les conditions sont réunies (True)
        return True
```

```
# Fonction qui exécute l'action (déplace les jetons)
def fire(self): 2 usages
    # 1. On "avale" (soustrait) les jetons des places d'entrée
    for place, weight in self.input_arcs.items():
        place.tokens -= weight
    # 2. On "recrache" (additionne) les jetons dans les places de sortie
    for place, weight in self.output_arcs.items():
        place.tokens += weight
```


Traduction métier de l'EVC

Processus : Instanciation de la classe PetriNet

Logique : 9 places (États du train) et 16 transitions (Aléas et Sécurité)

Sécurité : Codage des arcs Fail-Safe (mémoire de panne incluse)


```
# TEST PROPRIÉTÉ P3 : Vérifie les invariants matériels
def check_invariant(self, coeffs: Dict[str, int], constant: int) -> bool: 4 usages (2 dynamic)
    state_space, _ = self.build_state_space()
    sorted_places = sorted(self.places.keys())
    # Pour tous les états possibles, on fait la somme des jetons pour vérifier qu'elle égale la constante (ex: 1)
    for state in state_space:
        total = 0
        for i, p in enumerate(sorted_places):
            total += coeffs.get(p, default: 0) * state[i]
        if total != constant:
            return False # Dès qu'un invariant est brisé, on renvoie une erreur
    return True
```

```
def build_railway_braking_net() -> PetriNet: 1 usage
    net = PetriNet() # On fabrique le plateau vierge

    # --- 1. Création des Places (Les États du système ERTMS) ---
    net.add_place(name: "Marche_Normale", initial_tokens: 1)
    net.add_place(name: "Alerte_Vitesse", initial_tokens: 0)
    net.add_place(name: "Freinage_Service", initial_tokens: 0)
    net.add_place(name: "Freinage_Urgence", initial_tokens: 0)
    net.add_place(name: "Train_Arrete", initial_tokens: 0)
    net.add_place(name: "Radio_OK", initial_tokens: 1)
    net.add_place(name: "Radio_Perdue", initial_tokens: 0)
    net.add_place(name: "Odometrie_OK", initial_tokens: 1)
    net.add_place(name: "Derive_Odo", initial_tokens: 0)

    # --- 2. Création des Transitions (Les Actions) ---
    transitions = [
        "T_Survitesse", "T_Conducteur_Ralentit", "T_Ignorer_Alerte", "T_Echec_Service",
        "T_Arret_Complet", "T_Reprise_Marche",
        "T_Perte_Liaison", "T_Reparation_Radio",
        "T_Erreur_Capteur", "T_Recalage_Balise",
        "T_Urgence_Radio_Marche", "T_Urgence_Radio_Alerte", "T_Urgence_Radio_Service",
        "T_Urgence_Odo_Marche", "T_Urgence_Odo_Alerte", "T_Urgence_Odo_Service"
    ]
    for t in transitions:
        net.add_transition(t)
```


Algorithme BFS (Model Checking)

Objectif : Validation exhaustive (Norme SIL 4)

Méthode : Parcours en largeur (BFS) via `build_state_space()`

Résultat : Cartographie du "multivers" (24 états uniques)


```

def build_state_space(self) -> Tuple[Set[Tuple[int, ...]], Dict[Tuple[int, ...], List[Tuple[int, ...]]]: 3 usages (1 dynamic)
    self.reset()
    start = self.get_marking_tuple()
    visited = set() # Un ensemble pour retenir les états qu'on a déjà explorés
    queue = deque([start]) # Une file d'attente pour explorer le réseau façon "tache d'huile" (Parcours BFS)
    graph = {} # Le dictionnaire qui va contenir l'arbre de tous les chemins possibles

    while queue:
        state = queue.popleft() # On prend un état dans la file
        if state in visited:
            continue # Si on le connaît déjà, on passe au suitant
        visited.add(state) # On marque cet état comme "connu"
        self.set_marking(state) # On place physiquement les jetons du réseau dans cet état pour le tester

        succ_states = [] # Liste pour retenir où on peut aller depuis cet état
        for trans in self.transitions.values():
            if trans.is_enabled():
                # Petite astuce : on sauvegarde les jetons avant de tester la transition
                old_tokens = {p: p.tokens for p in trans.input_arcs.keys() | trans.output_arcs.keys()}
                trans.fire() # On teste l'action
                new_state = self.get_marking_tuple() # On mémorise le résultat
                succ_states.append(new_state)
                # On annule l'action (on remet les jetons comme avant) pour pouvoir tester les autres transitions
                for p, tokens in old_tokens.items():
                    p.tokens = tokens
                # Si ce nouveau futur est inconnu, on l'ajoute à la file d'attente pour l'explorer plus tard
                if new_state not in visited:
                    queue.append(new_state)

        graph[state] = succ_states # On relie l'état à tous ses futurs possibles dans le graphe
    return visited, graph

```


Vérification automatique des propriétés

Fonctions d'interrogation :

- **check_deadlock** : Pas de "cul-de-sac"
- **check_liveness** : Aucune transition "morte"
- **check_bornitude** : Aucun engorgement mémoire
- **check_invariant** : Somme des ressources toujours = 1


```
# TEST PROPRIÉTÉ P5 : Vérifie l'absence d'interblocage (Deadlock)
```

```
def check_deadlock(self, state_space: Set[Tuple[int, ...]], 2 usages (1 dynamic)  
    graph: Dict[Tuple[int, ...], List[Tuple[int, ...]]]) -> bool:
```

```
# On fouille tous les états de l'univers possible
```

```
for state in state_space:
```

```
    # Si un état n'a aucun futur (graph[state] est vide), c'est un Deadlock.
```

```
    if not graph[state]:
```

```
        return False
```

```
return True # Si on a tout vérifié sans rien trouver, le système est "Deadlock-free"
```

```
# TEST PROPRIÉTÉ P3 : Vérifie les invariants matériels
```

```
def check_invariant(self, coeffs: Dict[str, int], constant: int) -> bool: 4 usages (2 dynamic)
```

```
    state_space, _ = self.build_state_space()
```

```
    sorted_places = sorted(self.places.keys())
```

```
# Pour tous les états possibles, on fait la somme des jetons pour vérifier qu'elle égale la constante (ex: 1)
```

```
for state in state_space:
```

```
    total = 0
```

```
    for i, p in enumerate(sorted_places):
```

```
        total += coeffs.get(p, default: 0) * state[i]
```

```
    if total != constant:
```

```
        return False # Dès qu'un invariant est brisé, on renvoie une erreur
```

```
return True
```


Synthèse des Résultats

Propriété Vérifiée	Cible / Formule	Outil	Résultat
P1. Sécurité	$\mathbf{G}(\neg(\text{Marche} \wedge \text{Freinage}))$	Python	VÉRIFIÉE
P2. Réactivité	$\mathbf{G}((\text{Marche} \wedge \text{Radio_P}) \rightarrow \mathbf{F} \text{ Freinage})$	Python	VÉRIFIÉE
P3. Invariants	Radio_OK + Radio_Perdue = 1	Python	VÉRIFIÉE
P4. Vivacité	$\mathbf{G}(\text{Freinage} \rightarrow \mathbf{F} \text{ Train_Arrete})$	Python	VÉRIFIÉE
P5. Absence de Deadlock	$\mathbf{AG}(\mathbf{EX} \text{ true})$	Python	VÉRIFIÉE
P6. Bornitude	$\forall p, M(p) \leq 1$	TINA	VÉRIFIÉE

TABLE 2 – Synthèse des résultats de la vérification formelle

Preuve de Robustesse et Limites

C:\Users\sirin\Documents\JNG1_GM\S2\Projet\RDP.ktz

digest	places	9	transitions	16	net	bounded	Y	live	?	reversible	?
	abstraction		count		props		psets		dead		live
help	states		20		9		20		?		?
	transitions		76		16		16		?		?

```
state 0
props Marche_Normale Odometrie_OK Radio_OK
trans T_Erreur_Capteur/1 T_Perte_Liaison/2 T_Survitesse/3

state 1
props Derive_Odo Marche_Normale Radio_OK
trans T_Recalage_Balise/0 T_Perte_Liaison/4 T_Survitesse/5 T_Urgence_Odo_Marche/6

state 2
props Marche_Normale Odometrie_OK Radio_Perdue
trans T_Reparation_Radio/0 T_Erreur_Capteur/4 T_Survitesse/7 T_Urgence_Radio_Marche/8

state 3
props Alerte_Vitesse Odometrie_OK Radio_OK
trans T_Conducteur_Ralentit/0 T_Erreur_Capteur/5 T_Perte_Liaison/7 T_Ignorer_Alerte/9

state 4
props Derive_Odo Marche_Normale Radio_Perdue
trans T_Reparation_Radio/1 T_Recalage_Balise/2 T_Survitesse/10 T_Urgence_Radio_Marche/11 T_Urgence_Odo_Marche/11

state 5
props Alerte_Vitesse Derive_Odo Radio_OK
trans T_Conducteur_Ralentit/1 T_Recalage_Balise/3 T_Urgence_Odo_Alerte/6 T_Perte_Liaison/10 T_Ignorer_Alerte/12

state 6
props Derive_Odo Freinage_Urgence Radio_OK
trans T_Perte_Liaison/11 T_Arret_Complet/13 T_Recalage_Balise/14

state 7
props Alerte_Vitesse Odometrie_OK Radio_Perdue
trans T_Conducteur_Ralentit/2 T_Reparation_Radio/3 T_Urgence_Radio_Alerte/8 T_Erreur_Capteur/10 T_Ignorer_Alerte/15

state 8
props Freinage_Urgence Odometrie_OK Radio_Perdue
trans T_Erreur_Capteur/11 T_Reparation_Radio/14 T_Arret_Complet/16

state 9
props Freinage_Service Odometrie_OK Radio_OK
trans T_Erreur_Capteur/12 T_Echo_Service/14 T_Perte_Liaison/15

state 10
props Alerte_Vitesse Derive_Odo Radio_Perdue
trans T_Conducteur_Ralentit/4 T_Reparation_Radio/5 T_Recalage_Balise/7 T_Urgence_Radio_Alerte/11 T_Urgence_Odo_Alerte/11 T_Ignorer_Alerte/17

state 11
props Derive_Odo Freinage_Urgence Radio_Perdue
trans T_Reparation_Radio/6 T_Recalage_Balise/8 T_Arret_Complet/18
```


Conclusion et Perspectives

Bilan technique :

- **Modèle robuste** : Aucun blocage (Deadlock-free)
- **Preuve SIL 4** : Système stable et 1-borné

Limites identifiées :

- Modèle logique abstrait (pas de gestion du temps réel)
- Simplification des variables physiques

Perspectives :

- Intégration d'Automates Temporisés (type UPPAAL).
- Validation des délais de réaction physiques à 300 km/h.